The Journal of Systems and Software 231 (2026) 112591

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

An alternative software benchmarking dataset: effort estimation with machine learning

Ozan Raşit Yürüm [a,*], Hüseyin Ünlü [b], Onur Demirörs [b]

[a] Department of Computer Engineering, İzmir Bakırçay University, İzmir 35665, Türkiye
[b] Department of Computer Engineering, İzmir Institute of Technology, İzmir 35430, Türkiye

ARTICLE INFO

Editor: Professor Laurence Duchien

Keywords:
Size measurement
Effort estimation
COSMIC
ISBSG
Machine learning
Dataset
Benchmarking

ABSTRACT

Effort estimation plays a vital role in software project planning, as accurate estimates of required human resources are essential for success. Traditional estimation models often depend on historical size and effort data, yet organizations frequently struggle to access reliable effort records. Public benchmarking datasets like ISBSG offer useful data but may lack coverage or involve licensing fees. To address this issue, we previously introduced a free, extendable benchmarking dataset that integrates functional size and effort data extracted from 18 studies. In this study, we examine the effectiveness of our dataset for predictive effort estimation and compare it with the widely used ISBSG dataset. Our analysis includes 337 records from our dataset and 732 ISBSG projects, focusing on those with COSMIC size data. We first developed and compared models using linear regression and nine machine learning algorithms — Bayesian Ridge, Ridge Regression, Decision Tree, Random Forest, XGBoost, LightGBM, k-Nearest Neighbors, Multi-Layer Perceptron, and Support Vector Regression. Then, we selected the best-performing models and applied them to an unseen evaluation dataset to assess their generalization performance. The results show that machine learning performance varies based on evaluation method and dataset characteristics. Despite having fewer records, our dataset enabled more accurate predictions than ISBSG in most cases, highlighting its potential for effort estimation. This study demonstrates the viability of our dataset for building predictive models and supports the use of machine learning in improving estimation accuracy. Expanding this dataset could offer a valuable, open-access resource for organizations seeking effective and low-cost estimation solutions.

1. Introduction

A software development project is considered successful when it fulfills its customers' expectations and is delivered on the agreed time and budget (Standish Group Chaos Report, 2020). Therefore, in project management, effort estimation is important in accurately predicting the required human resource, which is the most fundamental resource in software projects (Agrawal and Chari, 2007; Gencel and Demirors, 2008). In most cases, two primary inputs are utilized to create an effort prediction model: historical size and effort data. The size of the software project can be measured at different stages of the project, from the requirements, code, or user interface. However, accessing historical effort data from previous projects may not always be possible within organizations. Therefore, it can be challenging to create an estimation model (Özkaya et al., 2011). In such cases, software organizations may use software benchmarking datasets to construct effort estimation models or organizational benchmarking purposes.

There are different software benchmarking datasets, including different numbers of records, attributes, and units of size measure (Bosu and Macdonell, 2019). To illustrate, the International Software Benchmarking Standards Group (ISBSG) (ISBSG Repository, 2021), one of the most comprehensive datasets, includes 11,128 development and enhancement projects from 26 countries. The four main functional sizing methods represented in the repository are IFPUG CPM 4.0 (ISO/IEC 20926, 2009) or later, COSMIC (ISO/IEC 19761, 2017), FiSMA (ISO/IEC 29881, 2010), and NESMA (ISO/IEC 24570, 2018). There are different studies in which effort prediction is performed using the ISBSG dataset (González-Ladrón-de-Guevara et al., 2016; Mendes et al., 2005; Ünlü et al., 2021). The data in these benchmark sets usually do not have enough details, such as architecture and the technologies utilized, and have notable variances of productivity that make it less useful for establishing prediction models. ISBSG also incurs a fee

* Corresponding author.
E-mail addresses: ozan.yurum@bakircay.edu.tr (O.R. Yürüm), huseyinunlu@iyte.edu.tr (H. Ünlü), onurdemirors@iyte.edu.tr (O. Demirörs).

https://doi.org/10.1016/j.jss.2025.112591
Received 2 March 2025; Received in revised form 24 June 2025; Accepted 31 July 2025
Available online 5 August 2025
0164-1212/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(Fernández-Diego and González-Ladrón-de-Guevara, 2014).

The general approach to effort estimation in the literature is to filter projects from an organization's past projects or benchmarking datasets like ISBSG and build a regression-based model. Many studies have developed predictive effort estimation models using this approach, and software organizations have achieved success in project planning with the regression-based models they created (Abran et al., 2015; Hacaloğlu et al., 2024; Salmanoglu et al., 2017; Ünlü et al., 2023). However, in today's software world, machine learning algorithms, which are used for various purposes (Al-Sabbagh et al., 2022; Bergström et al., 2022), have a huge potential to perform prediction tasks (Giray, 2021; Polkowski et al., 2019; Zhang and Tsai, 2003). These algorithms can provide a more precise estimation of effort.

In the literature, different studies attempted to evaluate the success of prediction models using various functional size measures as input. However, to the best of our knowledge, no study has gathered the functional size and effort data from the literature. In our previous study (Yürüm et al., 2024), we aimed to answer whether creating a useful software benchmark dataset is possible by gathering the data from the literature, thus addressing the gaps in existing datasets. We formed a dataset including 337 records from 18 studies that shared the related size and effort data. Although we performed a limited search, we created a larger dataset than many datasets (excluding ISBSG) in the literature (Bosu and Macdonell, 2019). In summary, we provided a reliable, free, and extendable software benchmarking dataset derived from scientific studies.

Although benchmarking datasets serve various purposes, they are primarily used for effort estimation when an organization lacks historical data. Numerous studies have demonstrated the effectiveness of the ISBSG dataset in this context. However, can the dataset we created also support predictive effort estimation? This study aims to answer that question. Building on the promising results of our previous study, where we explored the feasibility of constructing a software benchmarking dataset through a systematic literature review, we expanded our research with two key questions: (1) Can our dataset enable predictive effort estimation? (2) How can a machine-learning-based approach improve estimation precision?

For this purpose, we developed effort estimation models using 337 projects from our dataset and 732 projects from the ISBSG repository that include COSMIC size data. We categorized productivity levels as high and low, dividing the data into two separate groups. For each group, in addition to traditional linear regression, we applied nine machine learning algorithms: Bayesian Ridge, Ridge Regression, Decision Tree, Random Forest, XGBoost, LightGBM, k-Nearest Neighbors (KNN), Multi-Layer Perceptron (MLP), and Support Vector Regression (SVM). These algorithms represent a broad range of predictive approaches, including linear, tree-based, instance-based, neural network-based, and kernel-based methods. After training and validating the models using LOOCV, we selected the best-performing algorithms and evaluated their generalization ability on unseen test data.

The results demonstrated that machine learning algorithms enabled better predictive effort estimation. While the effort estimation model built with our dataset did not always yield a statistically significant change, it provided better predictive results compared to the model based on the ISBSG dataset. The findings of this extended study further validate the dataset's potential. Despite having fewer projects than ISBSG, it facilitated a better predictive effort estimation model. Expanding the dataset could further enhance domain-specific models and offer organizations free access to a valuable resource for effort estimation.

In summary, the contributions of this study are as follows. First, it builds upon our previous study, where we created a free, extendable software benchmarking dataset derived from 337 records across 18 studies. Second, it demonstrates that this dataset can be used for predictive effort estimation, showing its potential compared to the ISBSG dataset. Third, the study applies a variety of machine learning algorithms, including tree-based and neural network-based methods, to improve the accuracy of effort estimation over traditional linear regression models.

The paper is structured as follows: Section 2 covers the related work, Section 3 details the research methodology, Section 4 presents the results, Section 5 discusses findings, including validity threats, and Section 6 concludes with suggestions for further studies.

2. Related work

We summarize the related work in the literature in two main categories: using ISBSG for effort estimation purposes and using machine learning in the effort estimation of software projects.

Organizations often rely on ISBSG data to build effort estimation models when they lack sufficient historical data to create internal models. Several studies have highlighted the use of ISBSG for effort estimation in such cases. Ünlü et al. (2021) demonstrated the applicability of ISBSG data for effort estimation by comparing the estimated and actual effort of projects, revealing an average error rate of around 15 %. Mendes et al. (2005) compared effort estimation models using both organizational historical data and ISBSG data, finding similar error rates for both. Özcan-Top et al. (2011) assessed the effectiveness of external and multi-organizational software benchmarking repositories, concluding that ISBSG provided the most accurate results after the organization's own data.

However, like many software engineering datasets, the ISBSG dataset suffers from missing data, which can affect the accuracy of estimation models. To address this, Bala and Abran (2018) applied simple and multiple imputation techniques, finding that simple imputation with a relative average seed produced the most accurate estimates compared to a complete dataset. Finally, González-Ladrón-de-Guevara et al. (2016) conducted a literature review of 107 articles using ISBSG data for effort estimation, identifying size and effort as the most frequently used variables, with data quality ranking and counting approach variables often used for filtering.

Machine learning algorithms have great potential for prediction tasks, making it important to assess popular and effective algorithms for software effort estimation. These algorithms have been widely used in various domains for prediction and classification (Afuwape et al., 2021; Kolukisa and Bakir-Gungor, 2023; Teles et al., 2021). In the field of software engineering, several studies have examined different machine learning algorithms for effort estimation, producing varying results depending on the datasets and algorithms used (Brar and Nandal, 2022).

BaniMustafa, 2018 applied Random Forest for software effort estimation and found that it achieved higher accuracy than traditional methods. Goyal and Bhatia (2019) showed that Multi-Layer Perceptron performed better than linear regression for effort estimation. Lowast and Kumari, 2020 compared machine learning algorithms and concluded that Support Vector Machines provided the most accurate results . Rehman et al., 2021 analyzed several machine learning methods and found that Decision Trees gave better results in most cases, while Ridge Regression worked better in some situations . Sahoo et al., 2022 studied multiple regression techniques and found that Bayesian Ridge Regression was the most accurate compared to Decision Tree, Support Vector Regression, and XGBoost. Natarajan and Balachandran (2022) proposed an approach using XGBoost that considered risk factors and improved effort estimation accuracy. Rahman et al. (2023) tested several machine learning models and found that Decision Trees performed better than other algorithms such as k-Nearest Neighbors and Support Vector Regression. Iordan and Covaciu (2023) examined k-Nearest Neighbors and found it useful for predicting effort. Demir et al., 2023 investigated LightGBM and confirmed its reliability for effort estimation . Venkatraman et al. (2024) showed that Random Forest had better performance than other ensemble learning methods. Lastly, Ünlü et al., 2025 compared linear regression with six machine learning algorithms for data warehouse projects in the ISBSG repository and found that Decision

Tree and XGBoost outperformed the other models.

Recent studies have demonstrated that various machine learning algorithms can provide accurate effort estimations. However, they also highlight that there is no one-size-fits-all solution, as the most effective algorithm can vary depending on the specific problem at hand. In this study, in addition to linear regression, we applied nine machine learning algorithms: Bayesian Ridge, Ridge Regression, Decision Tree, Random Forest, XGBoost, LightGBM, k-Nearest Neighbors (KNN), Multi-Layer Perceptron (MLP), and Support Vector Regression (SVM). These algorithms represent a broad range of predictive approaches, including linear, tree-based, instance-based, neural network-based, and kernel-based methods.

3. Research methodology

The research methodology of this study consists of two main steps: the construction of the dataset conducted in our previous study and effort estimation with machine learning carried out in this study.

3.1. Construction of the dataset

In the construction of the benchmarking dataset step, we followed the standard process for conducting a systematic literature review (Kitchenham, 2004). The details of the steps performed can be found in our previous study (Yürüm et al., 2024). The process involved defining research objectives and questions, performing pilot searches, and constructing a search string focused on COSMIC size measurement. We used Scopus as the primary database and applied specific inclusion and exclusion criteria to filter relevant studies. An initial search yielded 548 publications, which were refined through title, abstract, and full-text screening, ultimately selecting 18 studies. Data extraction and synthesis were conducted collaboratively, ensuring accuracy and consistency for further analysis.

3.2. Effort estimation with machine learning algorithms

Effort estimation includes the procedures employed to develop machine learning models for software effort estimation. These procedures are explained in five subsections: data preprocessing, model development, model evaluation, performance indicators and statistical comparison.

3.2.1. Data preprocessing

Prior to model development, rigorous preprocessing steps were applied to ensure data quality and suitability for machine learning:

- Projects were segmented into low-productivity and high-productivity groups based on mean productivity values.
- Logarithmic transformation was applied to functional size and effort to improve distributional symmetry and compress scale.
- Outliers were identified and removed to retain only representative project data and enhance model reliability.

3.2.2. Model development

A comprehensive suite of ten machine learning algorithms was implemented to predict software effort. The selection encompassed a broad spectrum of techniques, including:

- Linear Approaches: Linear Regression, Bayesian Ridge, and Ridge Regression
- Tree-Based Methods: Decision Tree, Random Forest, XGBoost and LightGBM
- Instance-Based Learning: k-Nearest Neighbors (KNN)
- Neural Networks: Multi-Layer Perceptron (MLP)
- Kernel-Based Methods: Support Vector Regression (SVM)

Model training was performed using a Leave-One-Out Cross-Validation (LOOCV) scheme, which maximized the use of available data and provided an unbiased assessment of predictive performance.

3.2.3. Model evaluation

Following model development, model evaluation was performed using a dedicated subset of held-out project instances, referred to as the shared evaluation dataset. The construction of this dataset is detailed in Section 4.2. All entries in the shared set were excluded from training and validation to ensure unbiased assessment of generalization performance.

After identifying the best-performing model for each productivity group via Leave-One-Out Cross-Validation (LOOCV), these models were applied to the shared dataset. No further training or tuning was conducted during this evaluation phase. This procedure allowed us to assess how well the selected models transferred to previously unseen data under consistent conditions.

Performance results obtained from this evaluation were used in conjunction with the metrics described in Section 3.2.4 and subjected to statistical comparison as outlined in Section 3.2.5.

3.2.4. Performance indicators

We used widely accepted indicators from the literature to evaluate the prediction accuracy of the effort estimation models: (1) Magnitude of Relative Error (MRE), (2) Mean Magnitude of Relative Error (MMRE), (3) Median Magnitude of Relative Error (MdMRE), and (4) Percentage Relative Error Deviation within X (PRED(X)) (Runeson and Höst, 2008). MRE serves as the foundation for the other indicators.

$$\mathrm{MRE} = \left|\frac{\text{Actual Effort} - \text{Estimated Effort}}{\text{Actual Effort}}\right| \tag{1}$$

Various threshold values have been proposed for MMRE (Conte et al., 1986; Hastings and Sajeev, 2001), with Hastings and Sajeev (2001) suggesting that models with an MMRE below 0.20 are predictive, those between 0.20 and 0.50 are acceptable, and those above 0.50 are not acceptable. While MMRE is commonly used, it is often criticized for its sensitivity to high MRE values (Foss et al., 2003; Jorgensen, 1995; Jørgensen et al., 2022; Kitchenham et al., 2001; Myrtveit et al., 2005. In such cases, MdMRE, which is less influenced by extreme values, can be a better alternative (Jeffery et al., 2000).

$$\mathrm{MMRE} = \frac{1}{n}\sum_{i=1}^{n}\mathrm{MRE}_i \tag{2}$$

For PRED(X), "n" represents the total number of observations, and "k" denotes those with an MRE below "x". Among PRED values, PRED (0.25) and PRED(0.30) are the most frequently used in the literature (MacDonell, 1997; van Koten and Gray, 2006).

$$\mathrm{PRED}(x) = \frac{k}{n} \tag{3}$$

3.2.5. Statistical comparison

Finally, we conducted a Wilcoxon Signed-Rank Test to compare the predictive performance of the top-performing models trained on our dataset and the ISBSG dataset. The Wilcoxon test is a nonparametric alternative to the paired *t*-test, suitable for evaluating differences between two related samples without assuming normality (Wilcoxon, 1945; Hollander, Wolfe, and Chicken, 2013). This makes it appropriate for our analysis, as the distribution of the differences between paired prediction errors deviates from normality. In our implementation, we used the *scipy.stats.wilcoxon* function from the SciPy library (Virtanen et al., 2020) to analyze the absolute prediction errors generated on the shared evaluation dataset. By comparing the residuals from models trained on each dataset, we assessed whether the observed differences in predictive performance were statistically significant.

4. Results

This section introduces the constructed dataset, and the shared evaluation dataset derived from ISBSG and our dataset, followed by the results of effort estimation using machine learning techniques.

4.1. Dataset

As a result of our systematic literature review, we formed a dataset including 337 records from 18 studies that shared the corresponding size and effort data. The following sections summarize the dataset and compare it with the ISBSG dataset.

4.1.1. Dataset summary

We constructed the dataset based on 11 features derived from the ISBSG repository features and the provided case details of the studies. As many studies lack some ISBSG features, we could not include all features of the ISBSG dataset. These features are demonstrated in Table 1, which includes their descriptions and some examples.

In the distribution of development types in the dataset, most studies used effort and size data from newly developed projects, while enhancement projects were less common. In terms of effort units, person-hour was the most commonly used, followed by person-day in nearly half of the studies and person-month in a few. Due to the variation in effort units, normalized effort, defined as person-hour, was derived. Additionally, each project's domain/type was analyzed, revealing that most projects were categorized as management information systems or application systems, and the dataset also included projects from various domains such as telecommunication, finance, banking, and defense.

Productivity is defined as the ratio of normalized effort (in person-hours) to functional size (in COSMIC Function Points. Although productivity is generally defined in the literature as output/input (e.g., FP/hour), in this study we followed the ISBSG convention and calculated productivity as input/output (e.g., hours/FP), also known as Project Delivery Rate (PDR). This approach was adopted to ensure consistency in comparison between our dataset and ISBSG. While PDR and productivity are mathematical inverses, they serve equivalent analytical purposes and are frequently reported under the same label in effort estimation studies.

Table 1
Dataset features.

Feature	Description	Example
Case no	Identifier of the empirical study or empirical case A study may include multiple cases.	1, 2
Project no	Identifier of the project within a given case. Each case may include multiple projects, and this number uniquely identifies them within that context.	1, 2, 3, 4, 5
Project domain	Project domain or type in a case	Management information system, defense industry
Development type	Information indicating whether a project was newly developed or built on an existing project	New development, enhancement
Implementation environment	The technology or language used to develop a project	Java, .Net, PHP, Azure DevOps
Cosmic size	Measured COSMIC function point size of a project	61, 77, 116, 715
Actual effort	Actual effort to complete a project	76, 89, 156, 8610
Original effort unit	The original unit of actual effort in a study	person-hour, person-day, person-month
Normalized effort	The value of actual effort in person-hour	608, 89, 1248, 8610
Productivity	Division of normalized effort by COSMIC size	9.07, 1.16, 10.76
SDLC scope	Software development life cycle activities included in the effort	Construction, implementation, management, maintenance

4.1.2. Comparison with ISBSG

We compared its descriptive statistics with those of the ISBSG - COSMIC dataset to assess the characteristics of our dataset. This comparison focuses on three key aspects: project size, effort, and productivity (see Table 2, Table 3, and Table 4).

Our dataset includes a variety of project sizes, with a smaller overall range compared to the ISBSG dataset. The average project size in our dataset is lower than those in the ISBSG repository. Furthermore, our dataset exhibits less variability in project size, suggesting a more consistent distribution of project types. When comparing effort, our dataset shows a lower average effort than the ISBSG dataset. This difference is accompanied by a smaller median and a less pronounced variation in effort, which reflects a more uniform distribution of effort across the projects in our dataset.

Regarding productivity, our dataset reveals less variation than the ISBSG repository. Projects in our dataset show a narrower range of productivity values, indicating that productivity is more concentrated around the average. This contrasts with the broader spread of productivity values in the ISBSG dataset, which may be influenced by the inclusion of larger and more complex projects. There may be two reasons behind this: (1) The ISBSG repository's mean size is greater than our dataset's mean size. Therefore, the complexity of large projects may result in low productivity, and that might increase the variation. (2) Many studies we used to create our dataset could have lacked support for SDLC activities, such as project management, configuration management, and reviews, including in the given effort. Therefore, if these projects only include partial SDLC activities, the reported effort may be lower than actual, resulting in higher productivity values.

4.2. Shared dataset from ISBSG and our dataset

To enable a fair and consistent comparison of model performance, we constructed a shared evaluation dataset comprising 19 unique project entries. These entries were selected based on matching values of the two core variables used in effort estimation: (1) Size and (2) Effort. Each matched pair was present in both our dataset and the ISBSG repository. The shared dataset was created by performing an inner join on these two variables across the datasets.

To prevent any form of data leakage during model training and evaluation, we removed all occurrences of these 19 shared entries from both datasets prior to model training. As a result, the filtered versions of our dataset and ISBSG included 315 and 704 projects respectively, comprising only non-overlapping observations. This approach ensured that the models trained on each dataset were evaluated on an identical set of previously unseen instances, allowing for a direct and statistically meaningful comparison of their generalization performance.

This shared evaluation dataset is later used for assessing model performance and comparing predictive outcomes between our dataset and ISBSG.

Table 2
Comparison of descriptive statistics - COSMIC size.

	Our Dataset	ISBSG - COSMIC
# of Records	337	732
Minimum	1	1
Maximum	2040	2839
Mean	136.30	188.95
Median	65	58
Standard Deviation	248.77	348.74

Table 3
Comparison of descriptive statistics - effort.

	Our Dataset	ISBSG - COSMIC
# of Records	337	732
Minimum	2.5	2.00
Maximum	20,712	47,493
Mean	1685.46	3028.68
Median	215	672.70
Standard Deviation	3279.38	5650.80

Table 4
Comparison of descriptive statistics - productivity.

	Our Dataset	ISBSG - COSMIC
# of Records	337	732
Minimum	0.03	0.20
Maximum	199.02	330.60
Mean	10.54	23.76
Median	4.33	14.40
Standard Deviation	17.83	34.92

4.3. Effort estimation with machine learning algorithms

Effort estimation using machine learning in our study is divided into two main stages: (1) Data Preprocessing and (2) Effort Estimation. In the following subsections, we detail each stage, outlining the individual steps and presenting the results obtained at each stage.

4.3.1. Data preprocessing

Data preprocessing is a crucial step in effort estimation as it ensures that the dataset is clean, well-structured, and suitable for machine learning models. This stage involves segmenting projects based on productivity levels, applying log transformation to normalize the data, and identifying and removing outliers that may seriously affect estimations. These preprocessing steps enhance data quality, reduce bias, and improve the predictive performance of the models.

4.3.1.1. Productivity-based division. Effort estimation accuracy can vary significantly based on the productivity levels of software development projects. To account for this variability, the dataset was segmented into two groups based on the mean productivity score. This segmentation allowed for the development of models that were tailored to the specific characteristics of each group, thereby improving predictive accuracy.

The productivity score for each project was calculated as the ratio of actual effort to functional size (CFP). A higher productivity score indicates that more effort was required per unit of functional size, while a lower score suggests greater efficiency in effort utilization. The mean productivity score of the dataset was used as a threshold to divide projects into the following categories:

1. Low-Productivity Projects: Projects with a productivity score higher than the mean, indicating greater effort per unit of functional size.
2. High-Productivity Projects: Projects with a productivity score lower than the mean, indicating more efficient effort utilization.

Once categorized, machine learning models were trained separately on each group. This approach enabled a more granular analysis of effort estimation patterns, allowing models to focus on the unique characteristics of projects within each productivity category. By distinguishing between different productivity levels, the models could better capture variations in software development effort, leading to more precise predictions.

The productivity-based division categorized 77 projects as low productivity and 238 projects as high productivity in our dataset. In contrast, ISBSG contained 210 low-productivity projects and 494 high productivity projects. Table 5 presents the descriptive statistics for low and high productivity levels in our dataset, while Table 6 provides the corresponding statistics for ISBSG.

Productivity plays a crucial role in effort estimation, with lower values indicating greater efficiency - meaning less effort is required to complete a given amount of work. When comparing Our Dataset with ISBSG, it becomes clear that our dataset consistently achieves better productivity, requiring less effort for similar tasks.

4.3.1.2. Log transformation. Software effort estimation datasets frequently exhibit right-skewed distributions, where a small number of projects require substantially higher effort than the majority. To address this issue, a natural logarithmic transformation was applied to both the CFP and Effort variables using NumPy's log function (Harris et al., 2020). This transformation significantly improved distributional symmetry and greatly stabilized variance, thereby enhancing the effectiveness of the model training process, particularly for linear models. By compressing extreme values and linearizing exponential relationships (Benoit, K., 2011), the transformation also facilitated more meaningful comparisons between linear and non-linear modeling approaches.

4.3.1.3. Outlier detection and removal. Although the log transformation reduced skewness and mitigated the influence of large values, it did not fully eliminate the effect of extreme cases in the datasets. Some data points continued to exhibit disproportionately high residuals or leverage after transformation. Therefore, additional outlier detection and removal were necessary to improve model stability and ensure more reliable training and evaluation.

Outliers can significantly impact the performance of machine learning models by introducing bias and distorting predictions. To ensure that our effort estimation datasets are both representative and free from extreme values, we implemented an outlier detection and removal process based on the Interquartile Range (IQR) method using Tukey's Hinges (Beyer, 1981). In this approach, any data point that falls >1.5 times the IQR below the first quartile (Q_1) or above the third quartile (Q_3) is flagged as an outlier.

The formulas used are:

Lower Fence $= Q_1 - 1.5 \times IQR$

Upper Fence $= Q_3 + 1.5 \times IQR$ where $IQR = Q_3 - Q_1$.

Based on this formula, one outlier was identified and removed from each of the low- and high-productivity project groups in our dataset. For the ISBSG dataset, eight low-productivity and three high-productivity projects were flagged as outliers and subsequently excluded from further analysis.

Table 5
Descriptive statistics for low and high productivity levels in our dataset.

	OUR DATASET - LOW			OUR DATASET - HIGH		
	Size	Effort	Productivity	Size	Effort	Productivity
# of Records	77	77	77	238	238	238
Minimum	2	32	10.76	3	2.50	0.03
Maximum	715	20,712	199,02	2040	9160	10.40
Mean	197.44	6112.83	32.51	127.31	389.18	3.70
Median	195	5819	25.07	53	144.50	2.80
Std. Deviation	125.56	4464.89	27.12	282.86	761.31	2.86

Table 6
Descriptive statistics for low and high productivity levels in ISBSG.

	ISBSG - LOW			ISBSG - HIGH		
	Size	Effort	Productivity	Size	Effort	Productivity
# of Records	210	210	210	494	494	494
Minimum	1	24	21.19	1	2	0.20
Maximum	2003	47,493	330.56	2839	28,358	20.80
Mean	150.68	5867.90	51.27	215.20	1597.31	8.26
Median	80	3180	33.93	52.50	240	6.97
Std. Deviation	223.99	7791.28	47.37	395.27	3500.53	5.79

The resulting dataset sizes after outlier removal were as follows:

- Our dataset: 76 low-productivity projects and 237 high-productivity projects
- ISBSG: 202 low-productivity projects and 491 high-productivity projects

Fig. 1 displays the boxplots generated using SPSS (Field, 2013), illustrating the identified outliers in each subgroup.

4.3.2. Model training and evaluation

Following data preprocessing, a Leave-One-Out Cross-Validation (LOOCV) strategy (Sreedharan et al., 2023) was used to train and validate machine learning models for effort estimation. LOOCV was chosen due to its ability to maximize training data utilization while providing an unbiased assessment of model performance. In this approach, each data instance was iteratively treated as the test set while the remaining data was used for model training.

A diverse set of machine learning models was employed for training, including Linear Regression, Bayesian Ridge, Ridge Regression, Decision Tree, Random Forest, XGBoost, LightGBM, k-Nearest Neighbors (KNN), Multi-Layer Perceptron (MLP), and Support Vector Regression (SVM). These models were selected to cover a broad range of predictive approaches, including linear, tree-based, instance-based, neural network-based and kernel-based methods. The entire modeling process was implemented in Python using established libraries and is summarized in the pseudocode provided in Fig. 2.

During each iteration of LOOCV, models were trained using the transformed training data and then used to predict the effort value for the test instance. The predictions were recorded, and the process continued until all instances in the dataset had been used as test cases. Once predictions were obtained, an inverse transformation was applied to restore the values to their original scale.

Model performance was evaluated using three widely recognized error metrics: Mean Magnitude of Relative Error (MMRE), Median Magnitude of Relative Error (MdMRE), and PRED(30). These metrics provided insights into the accuracy and reliability of each model, allowing for a comparative analysis of their effectiveness in effort

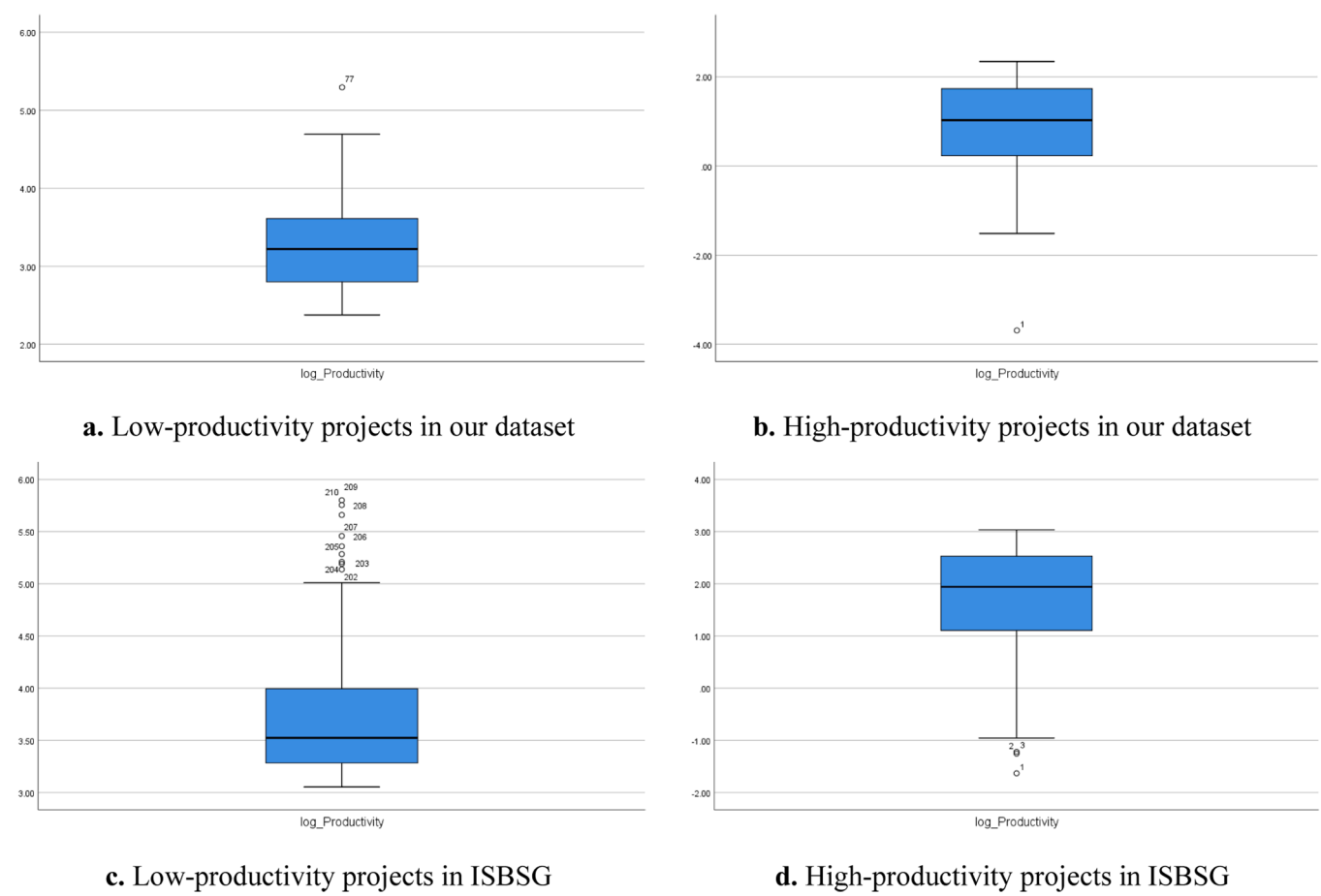

Fig. 1. Boxplots illustrating outliers in our dataset and ISBSG.

```
BEGIN

  IMPORT necessary libraries
  SET random seed
  LOAD preprocessed dataset

  # Dataset is divided into productivity groups: low and high
  # Log transformation and outlier removal were applied during preprocessing

  FOR each productivity group

        SELECT subset of dataset of current productivity group

        INITIALIZE machine learning models:
          - Linear Regression, Bayesian Ridge, Ridge
          - Decision Tree, Random Forest, XGBoost, LightGBM
          - k-Nearest Neighbors (KNN)
          - Multi-Layer Perceptron (MLP)
          - Support Vector Regression (SVM)

        INITIALIZE storage for actual and predicted values

        # --- LOOCV: Model Training and Validation ---
        FOR each instance in group dataset
              SPLIT dataset into training (excluding instance) and test (current instance)
              EXTRACT feature (log_CFP) and target (log_Effort)

              FOR each model in models
                    TRAIN model on training data
                    PREDICT log_Effort for test instance
                    STORE prediction

              STORE actual log_Effort

        APPLY back-transformation to actual and predicted values
        COMPUTE error metrics (MMRE, Pred(30), MdMRE) for each model
        SELECT best-performing model

        # --- Prediction on Shared (Unseen) Dataset ---
        LOAD shared dataset
        SELECT subset matching current productivity group
        APPLY log transformation to CFP

        PREDICT log_Effort using best performing model
        APPLY back-transformation to predicted value
        STORE predicted values for current group
        STORE actual Effort from shared dataset for this group

  END FOR

  # --- Final Evaluation on Shared (Unseen) Dataset ---
  COMBINE predictions from all productivity groups
  COMBINE actual values from shared dataset
  COMPUTE overall error metrics on shared dataset (MMRE, Pred(30), MdMRE)
  SAVE final predicted values and evaluation metrics

END
```

Fig. 2. Pseudocode for Effort Estimation.

estimation. Finally, all predicted values and computed error metrics were saved for further evaluation and reporting.

4.3.2.1. Model selection using our dataset. We evaluated effort estimation models for both low-productivity projects and high-productivity projects, as shown in Table 7 and Table 8, respectively.

Among the models trained and validated on low-productivity projects, Support Vector Regression (SVM) yielded the best results, achieving the lowest MMRE (0.478), the highest PRED(30) (0.481), and the lowest MdMRE (0.340). This indicates that SVM was the most accurate and robust model for this productivity group.

For high-productivity projects, LightGBM achieved the lowest MMRE (0.969), indicating the best average predictive performance, while KNN obtained the lowest MdMRE (0.553), suggesting more consistent and robust predictions across most instances. In terms of threshold-based accuracy, Random Forest (0.266) and XGBoost (0.262) yielded the highest PRED(30) values, reflecting a greater proportion of predictions within 30 % relative error. Although LightGBM had the best average error, its MdMRE and PRED(30) were moderate, whereas KNN combined strong typical-case accuracy with competitive performance on the other metrics. These findings highlight a trade-off between average and median accuracy, suggesting that KNN and LightGBM are the most promising models, depending on whether the application prioritizes robustness or overall error minimization.

4.3.2.2. Model selection using ISBSG. We also evaluated the same set of models on the ISBSG dataset, separating low-productivity and high-productivity projects. Table 9 summarizes results for low-productivity projects, while Table 10 shows results for high-productivity projects.

Among low-productivity projects, Support Vector Regression (SVM) consistently outperformed all other models, achieving the lowest MMRE (0.385), the highest PRED(30) (0.514), and the lowest MdMRE (0.295). Given that MdMRE was our primary selection criterion due to its robustness against extreme prediction errors, and SVM also led in the other two metrics, it was selected as the most effective model for this productivity group.

For high-productivity projects in the ISBSG dataset, MLP achieved the lowest MMRE (1.057), indicating superior average prediction accuracy, while Random Forest yielded the lowest MdMRE (0.538), reflecting better typical-case performance across most predictions. In terms of PRED(30), which measures the proportion of predictions within a 30 % error margin, SVM showed the best result (0.271), followed closely by Random Forest (0.265) and XGBoost (0.261). While no single model dominates across all three metrics, MLP and Random Forest stand out as strong candidates—MLP for its consistently low mean error, and Random Forest for its robustness and favorable balance between prediction accuracy and consistency. These results suggest that MLP may be preferable when minimizing average error is critical, whereas Random Forest may be more suitable when robustness and stable prediction accuracy are prioritized.

In summary, the results emphasize the importance of evaluating models using both MMRE and MdMRE, as these metrics capture different yet complementary aspects of estimation accuracy. MMRE reflects the average prediction error and is more sensitive to extreme values, making it suitable when minimizing total error is a priority. In contrast, MdMRE offers a more robust measure of typical-case performance, reducing the influence of outliers and highlighting consistency. For low-productivity projects, Support Vector Regression (SVM) consistently outperformed all other models in both our dataset and the ISBSG dataset, achieving the lowest MMRE and MdMRE values, and thus demonstrating superior average and robust performance. In the case of high-productivity projects, findings varied between datasets. In our dataset, LightGBM achieved the lowest MMRE, indicating strong average performance, while KNN yielded the lowest MdMRE, suggesting more stable typical-case predictions. In contrast, within the ISBSG dataset, MLP achieved the lowest MMRE, whereas Random Forest showed the lowest MdMRE. These results reveal that no single model consistently excels across both metrics or datasets for high-productivity projects. Therefore, model selection should be informed by whether the goal is to minimize overall estimation error or to prioritize robustness in typical prediction scenarios.

4.3.2.3. Evaluation on the shared unseen dataset. To assess the generalization capability of the models trained on our dataset, we conducted a comparative analysis with models trained on the ISBSG dataset. Using the Wilcoxon signed-rank test, we compared prediction results on a

Table 7
Comparison of models for low-productivity projects in our dataset.

Model	MMRE	PRED(30)	MdMRE
XGBoost	0.662	0.342	0.452
Linear Regression	0.478	0.382	0.403
MLP	0.571	0.342	0.504
Decision Tree	0.697	0.368	0.427
SVM	0.470*	0.461	0.320*
KNN	0.516	0.408	0.381
Random Forest	0.593	0.382	0.392
Bayesian Ridge	0.478	0.368	0.400
Ridge	0.479	0.368	0.398
LightGBM	1.230	0.368	0.516

Table 8
Comparison of models for high-productivity projects in our dataset.

Model	MMRE	PRED(30)	MdMRE
XGBoost	1.184	0.262	0.592
Linear Regression	1.072	0.207	0.612
MLP	1.122	0.215	0.606
Decision Tree	1.188	0.224	0.633
SVM	1.033	0.177	0.580
KNN	1.044	0.245	0.553*
Random Forest	1.068	0.266	0.623
Bayesian Ridge	1.071	0.207	0.602
Ridge	1.071	0.207	0.601
LightGBM	0.969*	0.219	0.594

Table 9
Comparison of models for low-productivity projects in ISBSG.

Model	MMRE	PRED(30)	MdMRE
XGBoost	0.580	0.424	0.380
Linear Regression	0.436	0.329	0.422
MLP	0.462	0.419	0.374
Decision Tree	0.627	0.410	0.370
SVM	0.385*	0.514	0.295*
KNN	0.463	0.410	0.358
Random Forest	0.542	0.433	0.368
Bayesian Ridge	0.436	0.329	0.421
Ridge	0.437	0.329	0.419
LightGBM	0.489	0.400	0.371

Table 10
Comparison of models for high-productivity projects in ISBSG.

Model	MMRE	PRED(30)	MdMRE
XGBoost	1.253	0.261	0.540
Linear Regression	1.082	0.248	0.551
MLP	1.057*	0.253	0.553
Decision Tree	1.304	0.224	0.585
SVM	1.216	0.271	0.544
KNN	1.137	0.232	0.584
Random Forest	1.189	0.265	0.538*
Bayesian Ridge	1.082	0.248	0.551
Ridge	1.082	0.248	0.551
LightGBM	1.088	0.230	0.546

shared unseen evaluation dataset. The comparisons were performed separately based on model selection using MdMRE and MMRE. The objective of this analysis was not to demonstrate that our dataset outperforms ISBSG, but rather to evaluate whether it can serve as a viable alternative for effort estimation tasks. The results, summarized in Table 11 and Table 12, provide insight into the predictive consistency and applicability of our dataset in comparison to an established benchmark.

Table 11 reports the comparison based on models selected using MdMRE. The model trained on our dataset achieved an MMRE of 0.540, an MdMRE of 0.560, and a PRED(30) of 0.263, whereas the ISBSG-based model yielded higher errors (MMRE: 1.150, MdMRE: 0.713) and a lower PRED(30) of 0.105. The difference between the two models was found to be statistically significant ($Z = 45$, $p = .045$), indicating better performance of the model trained on our dataset.

Table 12 presents the comparison based on models selected using MMRE. In this case, the model from our dataset again showed lower MMRE (0.520) and MdMRE (0.537) compared to the ISBSG model (MMRE: 0.689, MdMRE: 0.605). However, PRED(30) decreased in our dataset (0.105) compared to ISBSG (0.158), and the overall difference was not statistically significant ($Z = 68$, $p = .293$).

Overall, the results, whether statistically significant or not, indicate that our dataset can be effectively used to train effort estimation models. The consistent and competitive predictive performance observed on an unseen shared evaluation set supports the applicability of our dataset for effort estimation tasks, especially when appropriate model selection strategies are applied.

5. Discussion

In this section, we first address our research questions based on our findings and then examine potential threats to validity.

5.1. Answering research questions

In this study, building on our previous work, we formulated three main research questions: (1) To what extent can a software benchmarking dataset be created via a systematic literature review? (2) Can our dataset enable predictive effort estimation? (3) How can a machine-learning-based approach improve estimation precision?

5.1.1. To what extent can a software benchmarking dataset be created via a systematic literature review?

The findings of our previous study (Yürüm et al., 2024) indicated that a software benchmarking dataset could be constructed through a systematic literature review; however, the extent to which this was feasible was subject to several constraints. The dataset constructed in this study, consisting of 337 records from 18 studies that provided both COSMIC functional size and effort data, demonstrated that relevant project data could be systematically extracted from academic literature. Despite the search being limited to a single database, the resulting dataset was larger than many existing benchmarking datasets (Bosu and Macdonell, 2019), suggesting that leveraging published research was a viable approach for constructing software benchmarking datasets, particularly in cases where organizations lacked historical project data.

Nevertheless, several challenges were identified that impacted the completeness and applicability of such datasets. A key limitation was the restricted availability of project data within the literature. A significant proportion of studies (69 %) did not share their underlying datasets, instead reporting only evaluation indicators such as MMRE and PRED (n). This raised concerns regarding the transparency and reproducibility of research in the field of effort estimation. The inability to independently verify findings due to the absence of shared datasets posed a challenge to scientific rigor. While privacy considerations may have hindered data sharing, alternative approaches, such as anonymization, could have enhanced reproducibility while maintaining confidentiality.

Additionally, many studies failed to provide sufficient contextual details regarding their datasets. Essential attributes, including the implementation environment and the Software Development Life Cycle (SDLC) scope, were frequently omitted. The lack of such information diminished the practical utility of the dataset, as these details were critical for meaningful benchmarking and comparative analysis. Therefore, future research should emphasize the adoption of standardized reporting practices to ensure that datasets derived from literature are both reliable and comprehensive. In conclusion, while a systematic literature review facilitated the creation of a software benchmarking dataset, the quality and usability of the dataset were contingent upon the extent to which researchers shared their data and provided comprehensive descriptions. Despite all these limitations, we achieved to creation of a benchmarking dataset half the size of the COSMIC-sized projects in the most comprehensive ISBSG dataset by searching only in Scopus.

Table 11
Comparison of predictions on the shared unseen dataset (Best models selected based on MdMRE).

No	Dataset	N (Observations)	MMRE	MdMRE	PRED (30)	Z	p
1	Our dataset	19	0.540	0.560	0.263	45	.045
2	ISBSG	19	1.150	0.713	0.105		

Table 12
Comparison of predictions on the shared unseen dataset (Best models selected based on MMRE).

No	Dataset	N (Observations)	MMRE	MdMRE	PRED (30)	Z	p
1	Our dataset	19	0.520	0.537	0.105	68	.293
2	ISBSG	19	0.689	0.605	0.158		

5.1.2. Can our dataset enable predictive effort estimation?

Benchmarking datasets are widely used for various purposes, with one of their primary applications being effort estimation in cases where organizations lack historical data. While the ISBSG dataset has been extensively validated for this purpose, the predictive capability of our constructed dataset remained an open question. This study addressed this gap by assessing whether our dataset can support predictive effort estimation. To this end, we developed effort estimation models using 337 projects from our dataset and 732 COSMIC-sized projects from the ISBSG repository. Based on our previous experience (Ünlü et al., 2025), we divided projects into two sub-groups based on productivity: high and low productivity, and created separate effort estimation models for each group.

The primary objective of this study was to assess the effectiveness of predictive effort estimation models. It is widely recognized that developing an accurate effort estimation model using the ISBSG dataset depends heavily on proper project filtering. However, since our dataset is still in its early stages, we constructed effort estimation models for both the ISBSG dataset and our own dataset without applying any filtering, except for removing outlier projects and employing a productivity-based categorization.

The validation results confirmed that the constructed dataset is suitable for supporting predictive effort estimation. When applied to low-productivity projects, the dataset enabled consistent and reliable performance across multiple error metrics, suggesting that it provides a stable foundation for model development. In contrast, results for high-productivity projects showed greater variability depending on how performance was measured, indicating that outcomes in this category are more sensitive to data characteristics and evaluation metrics.

Compared to the ISBSG dataset, the constructed dataset showed comparable or better performance, particularly in scenarios where robustness was emphasized.

Further evaluation was conducted using a shared set of 19 previously unseen project instances. Models trained using this dataset demonstrated favorable generalization, especially when selected based on robust error measures. While performance differences were not always statistically significant under all selection criteria, the constructed dataset consistently enabled competitive predictive results. In some cases, it even outperformed the ISBSG dataset in both average and median error metrics, despite undergoing only minimal preprocessing.

In summary, the findings demonstrate that the constructed dataset holds strong potential for the development of accurate and generalizable effort estimation models. Its ability to support effective modeling across varying productivity levels and evaluation settings, without extensive filtering, underscores its value as a reliable resource. As the dataset continues to expand and evolve, its applicability to broader estimation tasks is expected to increase, making it a promising foundation for future research and practical applications.

5.1.3. How can a machine-learning-based approach improve estimation precision?

A machine-learning-based approach can really boost estimation accuracy by paying close attention to data preparation, validation, and the choice of modeling technique. By carefully cleaning the data - removing outliers, dividing projects into groups based on productivity, and applying a log transformation to normalize effort and size measurements - we ensure that the model is built on solid, representative data. Removing outliers helps eliminate extreme values that might otherwise distort predictions, while grouping projects by productivity allows the models to be tailored to the unique characteristics of low- and high-productivity projects. The log transformation further helps by smoothing out skewed data and making the estimation process more reliable.

Using a Leave-One-Out Cross-Validation (LOOCV) strategy is also a big advantage. LOOCV makes sure that every single data point is used for both training and validation, which helps prevent overfitting and gives a more honest picture of how well the model will perform on new data.

Machine learning models demonstrated strong predictive capabilities across both low- and high-productivity projects, with certain algorithms standing out depending on the productivity group and evaluation metric. For low-productivity projects, Support Vector Regression (SVM) consistently outperformed other models in both our dataset and the ISBSG dataset, yielding the lowest MMRE and MdMRE values and the highest PRED(30) scores. This highlights SVM's ability to provide accurate and robust predictions across varying data characteristics. For high-productivity projects, model performance was more context-dependent. In our dataset, LightGBM achieved the lowest MMRE, indicating strong average performance, while KNN produced the lowest MdMRE, suggesting more stable typical-case predictions. In contrast, within the ISBSG dataset, MLP and Random Forest were the most effective, excelling in average and median error, respectively.

Importantly, all evaluated machine learning models consistently outperformed traditional linear regression during Leave-One-Out Cross-Validation (LOOCV), across both productivity groups. This consistent advantage emphasizes the capacity of machine learning approaches to capture nonlinear patterns and interactions that traditional linear models may overlook. These findings underscore the importance of selecting models based on data characteristics and performance objectives and reinforce the value of machine learning techniques in advancing the accuracy and robustness of software effort estimation.

5.2. Threats to validity

Our study has several validity threats related to the research methodology followed during dataset construction and the application of machine learning algorithms for effort estimation. One limitation is that our search was conducted exclusively in the Scopus database, which may have led to the omission of relevant studies. However, before selecting Scopus, we performed pilot searches in other databases, including Web of Science, IEEE Xplore, ACM Digital Library, ScienceDirect, and Springer Link, and found that Scopus provided the most extensive coverage. Therefore, we chose it as our primary source. Additionally, we did not incorporate snowballing, a key practice in systematic literature reviews, within the study's scope. However, our primary objective was to explore whether a software benchmarking dataset could be constructed using existing literature. Thus, we focused on an exploratory search rather than an exhaustive one. Despite these limitations, our study successfully produced a dataset of substantial size.

Another threat may be related to eliminating outlier projects and splitting the data into two separate groups based on productivity while creating the effort estimation model. In the data preprocessing step of our study, we identified and eliminated outlier projects using statistical techniques such as the Interquartile Range (IQR) method to ensure that the dataset contained only meaningful and representative data points, improving the reliability and stability of the machine learning models used for effort estimation. When eliminating outlier projects in effort estimation modeling, maintaining an appropriate balance is crucial: removing too many projects may reduce the dataset size, thereby decreasing the model's reliability. However, in this study, the proportion of eliminated outlier projects was at most around 10 %, and since the dataset was sufficiently large, this was not a significant concern. After eliminating outlier projects, we leveraged our previous experience (Ünlü et al., 2025) to divide the data into two groups: low and high productivity. Since this categorization reduced the amount of data in the model, it was essential to ensure a balanced separation of both sub-groups. As a result, our dataset included 107 low-productivity projects and 189 high-productivity projects, while the ISBSG dataset contained 262 low-productivity projects and 415 high-productivity projects. Although the categorization was not perfectly balanced, the number of projects in each sub-group was still considerably high compared to many studies in the literature, allowing us to build reliable effort estimation models (Abran et al., 2015; Salmanoglu et al., 2017; Ünlü et al., 2021).

A further threat related to this productivity-based division is that it may raise concerns about bias or the requirement for prior knowledge. However, this classification was performed before model training and was applied independently within each dataset. The aim was to offer two baseline models that could help organizations without historical data select a suitable estimator based on project characteristics. In practice, organizations often have a reasonable understanding of whether a project is expected to require more or less effort, for example based on the domain, technological familiarity, or similarity to previously delivered projects. This intention has been explicitly clarified in the manuscript to ensure that the productivity split is not misinterpreted as part of the model evaluation procedure.

Even though our study's findings related to machine learning are promising, we recognize certain limitations that could be addressed in future research. Although the dataset compiled for this study includes multiple features, only functional size was used as a predictor of effort. This decision was driven by the limited availability and inconsistency of contextual information in many of the source studies. Essential attributes—such as implementation environment, development methodology, and the scope of SDLC activities—were frequently omitted or insufficiently described. This lack of standardized reporting made it impossible to identify a second independent variable that was both consistently available and shared with the ISBSG dataset. Consequently, our modeling relied on a single predictor across both datasets to ensure fairness and comparability.

Size is the most significant measurable aspect of software, and understanding its predictive power remains critical for effort estimation (Abran, 2015). Despite this limitation, machine learning models

produced better predictive performance than linear regression, suggesting their strength even under minimal feature conditions. As the software industry matures and develops new measures that are widely accepted and used in practice, there is no doubt that effort prediction models will continue to improve. To enable richer and more reliable predictive modeling in future studies, we recommend that empirical software engineering research adopt standardized data reporting practices, particularly when contributing to shared or reusable datasets.

Another limitation is our use of default parameters for all machine learning algorithms. This decision facilitated straightforward comparisons between models and datasets, thereby establishing a consistent baseline for our analyses. However, we acknowledge that machine learning models are highly sensitive to hyperparameter settings, and systematic hyperparameter tuning has the potential to significantly enhance prediction accuracy. Therefore, incorporating rigorous hyperparameter optimization in future work will not only reinforce our conclusion - that a machine-learning-based approach improves estimation precision - but may also yield even more accurate effort estimations. Overall, our study provides valuable insights into the effectiveness of machine learning for software effort estimation, and we anticipate that further refinement of these models will unlock their full potential.

Finally, it is important to clarify that the comparison of machine learning models in this study was based exclusively on performance obtained through Leave-One-Out Cross-Validation (LOOCV) using the training and validation data. Since no hyperparameter tuning was conducted, LOOCV served not only as a validation technique but also as the primary basis for assessing and comparing the performance of machine learning algorithms during model selection. While a shared dataset was used for evaluating the generalization capability of selected models, it did not serve as a basis for comparing all algorithms. This approach was intentionally adopted to prevent data leakage and ensure that model selection remained unbiased. As a result, all conclusions regarding model superiority are grounded in LOOCV-based validation results, which were used to identify the most consistent and accurate algorithms prior to final evaluation.

6. Conclusion

Effort estimation is crucial for software project planning, yet organizations often struggle due to the lack of reliable historical effort data. Benchmarking datasets, such as ISBSG, provide an alternative, but their applicability can be limited by access restrictions and data variability. To address this gap, we previously constructed a free and extendable software benchmarking dataset including 337 records from 18 studies that shared the related size and effort data. In this study, we investigated our dataset's effectiveness in predictive effort estimation and assess its performance against the widely used ISBSG dataset.

By applying a diverse set of machine learning algorithms, our study demonstrated that machine learning approaches offer strong predictive performance across both low- and high-productivity project groups. Support Vector Regression (SVM) consistently yielded the most accurate results for low-productivity projects in both our dataset and the ISBSG dataset. For high-productivity projects, no single algorithm was universally superior; LightGBM showed the best average performance in our dataset, while KNN produced more robust median predictions. In contrast, MLP and Random Forest emerged as top-performing models within the ISBSG dataset. These results highlight that the effectiveness of machine learning algorithms varies depending on dataset characteristics and project productivity levels, reinforcing the need to select models based on specific performance goals and data attributes.

Despite the promising results, some limitations remain. The dataset is still in its early stages, and expanding it with additional records could further enhance its reliability and applicability. Moreover, optimizing machine learning models through systematic hyperparameter tuning could improve their predictive accuracy. Although this study utilized only size and effort as variables in the models, this decision was made to ensure compatibility with ISBSG and due to the incomplete reporting of other features in the source studies. In that regard, future research would benefit from standardized and well-documented datasets that enable the inclusion of richer contextual variables.

Importantly, the main contribution of this study is not to outperform ISBSG in predictive performance, but to provide the community with an open-source benchmark dataset that supports empirical research on software effort estimation. Its predictive performance is only one aspect of its broader practical value. The comparison with ISBSG was intended to assess the initial potential of the dataset under a shared and realistic modeling setup. Moreover, the methodology adopted in this study—compiling data through a systematic literature review (SLR)—can be replicated to construct benchmark datasets in other areas of software engineering, thereby enabling broader and more diverse empirical investigations.

In conclusion, this study confirms that an alternative, freely available software benchmarking dataset can support predictive effort estimation. With continued expansion and refinement, this dataset has the potential to serve as a valuable resource for organizations that require reliable effort estimation models without incurring costs associated with proprietary datasets like ISBSG.

CRediT authorship contribution statement

Ozan Raşit Yürüm: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Hüseyin Ünlü:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Onur Demirörs:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Onur Demirörs reports financial support was provided by Scientific and Technological Research Council of Turkey. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported by The Scientific and Technological Research Council of Turkey (TUBITAK) ARDEB 1001 [Project number: 121E389] program. We would like to express our gratitude to all the scientists who contributed to creating this benchmarking dataset by openly sharing the data from their research.

Data availability

The dataset can be found at https://bit.ly/3Nhg5jy.

References

Abran, A., 2015. Software Project Estimation: The Fundamentals For Providing High Quality Information To Decision Makers. John Wiley & Sons Inc.

Abran, A., Desharnais, J.-M., Zarour, M., Demirörs, O., 2015. Productivity-based software estimation models and process improvement: an empirical study. Int. J. Adv. Softw 8, 103–114.

Afuwape, A.A., Xu, Y., Anajemba, J.H., Srivastava, G., 2021. Performance evaluation of secured network traffic classification using a machine learning approach. Comput. Stand. Interfaces 78, 103545. https://doi.org/10.1016/j.csi.2021.103545.

Agrawal, M., Chari, K., 2007. Software effort, quality, and cycle time: a study of CMM level 5 projects. IEEE Trans. Softw. Eng. 33, 145–156. https://doi.org/10.1109/TSE.2007.29.

Al-Sabbagh, K.W., Staron, M., Hebig, R., 2022. Improving software regression testing using a machine learning-based method for test type selection. In: Taibi, D.,

Kuhrmann, M., Mikkonen, T., Klünder, J., Abrahamsson, P. (Eds.), Product-Focused Software Process Improvement, Lecture Notes in Computer Science. Springer International Publishing, Cham, pp. 480–496. https://doi.org/10.1007/978-3-031-21388-5_33.

Bala, A., Abran, A., 2018. Impact analysis of multiple imputation on effort estimation models with the ISBSG repository of software projects. Softw. Meas. News 23, 17–34.

BaniMustafa, A., 2018. Predicting software effort estimation using machine learning techniques, in: 2018 8th International conference on computer science and information technology (CSIT). Presented at the 2018 8th International conference on computer science and information technology (CSIT), pp. 249–256. https://doi.org/10.1109/CSIT.2018.8486222.

Benoit, K., 2011. Linear regression models with logarithmic transformations. Lond. Sch. Econ. Lond. 22 (1), 23–36.

Bergström, G., Hujainah, F., Ho-Quang, T., Jolak, R., Rukmono, S.A., Nurwidyantoro, A., Chaudron, M.R.V., 2022. Evaluating the layout quality of UML class diagrams using machine learning. J. Syst. Softw. 192, 111413. https://doi.org/10.1016/j.jss.2022.111413.

Beyer, H., 1981. Tukey, John W.: exploratory data analysis. Addison-wesley publishing company reading, Mass. — Menlo Park, Cal., London, Amsterdam, Don Mills, Ontario, Sydney 1977, XVI, 688 S. Biom. J. 23, 413–414. https://doi.org/10.1002/bimj.4710230408.

Bosu, M.F., Macdonell, S.G., 2019. Experience: quality benchmarking of datasets used in software effort estimation. J. Data Inf. Qual. 11 (19), 38. https://doi.org/10.1145/3328746, 1-19.

Brar, P., Nandal, D., 2022. A systematic literature review of machine learning techniques for software effort estimation models, in: 2022 Fifth international conference on computational intelligence and communication technologies (CCICT). Presented at the 2022 fifth international conference on computational intelligence and communication technologies (CCICT), pp. 494–499. https://doi.org/10.1109/CCiCT56684.2022.00093.

Conte, S.D., Dunsmore, H.E., Shen, Y.E., 1986. Software Engineering Metrics and Models. Benjamin-Cummings Publishing Co., Inc., USA.

Demir, M.Ö., Gezici, B., Tarhan, A.K., 2023. Assessing the explainability of Lgbm Model for effort estimation on ISBSG dataset, in: 2023 4th international informatics and software engineering conference (IISEC). Presented at the 2023 4th International informatics and software engineering conference (IISEC), pp. 1–6. https://doi.org/10.1109/IISEC59749.2023.10391030.

Fernández-Diego, M., González-Ladrón-de-Guevara, F., 2014. Potential and limitations of the ISBSG dataset in enhancing software engineering research: a mapping review. Inf. Softw. Technol. 56, 527–544. https://doi.org/10.1016/j.infsof.2014.01.003.

Field, A., 2013. Discovering statistics using IBM SPSS statistics. Sage.

Foss, T., Stensrud, E., Kitchenham, B., Myrtveit, I., 2003. A simulation study of the model evaluation criterion MMRE. IEEE Trans. Softw. Eng. 29, 985–995. https://doi.org/10.1109/TSE.2003.1245300.

Gencel, C., Demirors, O., 2008. Functional size measurement revisited. ACM Trans. Softw. Eng. Methodol 17 (15), 1–15. https://doi.org/10.1145/1363102.1363106, 36.

Giray, G., 2021. A software engineering perspective on engineering machine learning systems: state of the art and challenges. J. Syst. Softw. 180, 111031. https://doi.org/10.1016/j.jss.2021.111031.

González-Ladrón-de-Guevara, F., Fernández-Diego, M., Lokan, C., 2016. The usage of ISBSG data fields in software effort estimation: a systematic mapping study. J. Syst. Softw. 113, 188–215. https://doi.org/10.1016/j.jss.2015.11.040.

Goyal, S., Bhatia, Pradeep.K., 2019. A non-linear technique for effective software effort estimation using multi-layer perceptrons, in: 2019 International conference on machine learning, big data, cloud and parallel computing (COMITCon). Presented at the 2019 international conference on machine learning, big data, cloud and parallel computing (COMITCon), pp. 1–4. https://doi.org/10.1109/COMITCon.2019.8862256.

Hacaloğlu, T., Ünlü, H., Yıldız, A., Demirörs, O., 2024. Software size measurement: bridging research and practice. IEEE Softw. 1–9. https://doi.org/10.1109/MS.2024.3358079.

Harris, C.R., Millman, K.J., Van Der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Oliphant, T.E., 2020. Array programming with NumPy. Nature 585 (7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2.

Hastings, T.E., Sajeev, A.S.M., 2001. A vector-based approach to software size measurement and effort estimation. IEEE Trans. Softw. Eng. 27, 337–350. https://doi.org/10.1109/32.917523.

Hollander, M., Wolfe, D.A., Chicken, E., 2013. Nonparametric Statistical Methods. John Wiley & Sons.

Iordan, A.-E., Covaciu, F., 2023. Estimation of the effort required tı develop a software through the K-nearest neighbours method. ACTA Tech. Napoc. - Ser. 66.

ISBSG, 2021. ISBSG Repository [WWW Document]. URL https://www.isbsg.org/ (accessed 10.18.21).

ISO/IEC, 2017. ISO/IEC 19761:2017 - Software Engineering – COSMIC: a functional size measurement method.

ISO/IEC, 2009. ISO/IEC 20926: 2009 - IFPUG functional size measurement method [WWW Document]. URL https://www.iso.org/standard/51717.html (accessed 6.6.23).

ISO/IEC, 2018. ISO/IEC 24570:2018 Software engineering — NESMA functional size measurement method [WWW Document]. URL https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/07/25/72505.html (accessed 6.6.23).

ISO/IEC, 2010. ISO/IEC 29881: 2010 - FiSMA 1.1 functional size measurement method [WWW Document]. URL https://www.iso.org/standard/56418.html (accessed 6.6.23).

Jeffery, R., Ruhe, M., Wieczorek, I., 2000. A comparative study of two software development cost modeling techniques using multi-organizational and company-specific data. Inf. Softw. Technol. 42, 1009–1016. https://doi.org/10.1016/S0950-5849(00)00153-1.

Jorgensen, M., 1995. Experience with the accuracy of software maintenance task effort prediction models. IEEE Trans. Softw. Eng. 21, 674–681. https://doi.org/10.1109/32.403791.

Jørgensen, M., Halkjelsvik, T., Liestøl, K., 2022. When should we (not) use the mean magnitude of relative error (MMRE) as an error measure in software development effort estimation? Inf. Softw. Technol. 143, 106784. https://doi.org/10.1016/j.infsof.2021.106784.

Kitchenham, B., 2004. In: Procedures for Performing Systematic Reviews, 33. Keele Univesity, Keele, UK, pp. 1–26.

Kitchenham, B.A., Pickard, L.M., MacDonell, S.G., Shepperd, M.J., 2001. What accuracy statistics really measure. IEE Proc. - Softw. 148, 81–85. https://doi.org/10.1049/ip-sen:20010506.

Kolukisa, B., Bakir-Gungor, B., 2023. Ensemble feature selection and classification methods for machine learning-based coronary artery disease diagnosis. Comput. Stand. Interfaces 84, 103706. https://doi.org/10.1016/j.csi.2022.103706.

Lowast, A.G.P.V., Kumari, K.A., 2020. Predictive analytics approaches for software effort estimation: a review. INDJST 13, 2094–2103. https://doi.org/10.17485/IJST/v13i21.573.

MacDonell, S.G., 1997. Establishing relationships between specification size and software process effort in CASE environments. Inf. Softw. Technol. 39, 35–45. https://doi.org/10.1016/0950-5849(96)01125-1.

Mendes, E., Lokan, C., Harrison, R., Triggs, C., 2005. A replicated comparison of cross-company and within-company effort estimation models using the ISBSG database. In: 11th IEEE International Software Metrics Symposium (METRICS'05). Presented at the 11th IEEE International Software Metrics Symposium (METRICS'05). https://doi.org/10.1109/METRICS.2005.4, p. 10 pp. –36.

Myrtveit, I., Stensrud, E., Shepperd, M., 2005. Reliability and validity in comparative studies of software prediction models. IEEE Trans. Softw. Eng. 31, 380–391. https://doi.org/10.1109/TSE.2005.58.

Natarajan, R., Balachandran, K., 2022. Ensemble model of machine learning for integrating risk in software effort estimation. In: Saraswat, M., Sharma, H., Balachandran, K., Kim, J.H., Bansal, J.C. (Eds.), Congress On Intelligent Systems. Springer Nature, Singapore, pp. 635–644. https://doi.org/10.1007/978-981-16-9416-5_46.

Özkaya, A., Ungan, E., Demirors, O., 2011. Common practices and problems in effort data collection in the software industry, in: 2011 joint conference of the 21st International workshop on software measurement and the 6th International conference on software process and product measurement. presented at the 2011 joint conference of the 21st International workshop on software measurement and the 6th International conference on software process and product measurement, pp. 308–313. https://doi.org/10.1109/IWSM-MENSURA.2011.40.

Polkowski, Z., Vora, J., Tanwar, S., Tyagi, S., Singh, P.K., Singh, Y., 2019. Machine learning-based software effort estimation: an analysis, in: 2019 11th international conference on electronics, computers and artificial intelligence (ECAI). Presented at the 2019 11th International conference on electronics, computers and artificial intelligence (ECAI), pp. 1–6. https://doi.org/10.1109/ECAI46879.2019.9042031.

Rahman, M., Roy, P.P., Ali, M., Gonc¸alves, T., Sarwar, H., 2023. Software effort estimation using machine learning technique. Int. J. Adv. Comput. Sci. Appl. (IJACSA) 14. https://doi.org/10.14569/IJACSA.2023.0140491.

Rehman, I.U., Ali, Z., Jan, Z., 2021. An empirical analysis on software development efforts estimation in machine learning perspective.

Runeson, P., Höst, M., 2008. Guidelines for conducting and reporting case study research in software engineering. Empir Softw. Eng. 14, 131. https://doi.org/10.1007/s10664-008-9102-8.

Sahoo, P., Behera, D.K., Mohanty, J.R., Kumar Dash, Ch.S., 2022. Effort estimation of software products by using UML sequence models with regression Analysis, in: 2022 OITS International conference on information technology (OCIT). Presented at the 2022 OITS International conference on information technology (OCIT), pp. 97–101. https://doi.org/10.1109/OCIT56763.2022.00028.

Salmanoglu, M., Hacaloglu, T., Demirors, O., 2017. Effort estimation for agile software development: comparative case studies using COSMIC functional size measurement and story points, in: proceedings of the 27th International workshop on software measurement and 12th International conference on software process and product measurement, IWSM Mensura '17. Association for computing machinery, Gothenburg, Sweden, pp. 41–49. https://doi.org/10.1145/3143434.3143450.

Sreedharan, R., Prajapati, J., Engineer, P., Prajapati, D., 2023. Leave-one-out cross-validation in machine learning. Ethical Issues in AI For Bioinformatics and Chemoinformatics. CRC Press.

Standish Group Chaos Report, 2020. . T.S.G. International.

Teles, G., Rodrigues, J.J.P.C., Rabêlo, R.A.L., Kozlov, S.A., 2021. Comparative study of support vector machines and random forests machine learning algorithms on credit operation. Software: practice and experience 51, 2492–2500. https://doi.org/10.1002/spe.2842.

Top, O.O., Ozkan, B., Nabi, M., Demirors, O., 2011. Internal and external software benchmark repository utilization for effort estimation, in: 2011 Joint conference of the 21st International workshop on software measurement and the 6th international conference on software process and product measurement. Presented at the 2011 joint conference of the 21st International workshop on software measurement and the 6th international conference on software process and product measurement, pp. 302–307. https://doi.org/10.1109/IWSM-MENSURA.2011.41.

Ünlü, H., Hacaloğlu, T., Ömüral, N.K., Çalişkanel, N., Leblebici, O., Demirörs, O., 2023. An exploratory case study on effort estimation in microservices. In: 2023 49th

Euromicro Conference on Software Engineering and Advanced Applications (SEAA). Presented at the 2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 215–218. https://doi.org/10.1109/SEAA60479.2023.00040.

Ünlü, H., Yalçın, A.G., Öztürk, D., Akkaya, G., Kalecik, M., Ekici, N.U., Orhan, O., Çiftçi, O., Yumlu, S., Demirörs, O., 2021. Software effort estimation using ISBSG dataset: multiple case studies, in: 2021 15th Turkish national software engineering symposium (UYMS). Presented at the 2021 15th Turkish national software engineering symposium (UYMS), pp. 1–6. https://doi.org/10.1109/UYMS54260.2021.9659655.

Ünlü, H., Yürüm, O.R., Yıldız, A., Demirörs, O., 2025. Application of a size measurement standard for Data warehouse projects. Pract. Exp. 55, 571–588. https://doi.org/10.1002/spe.3391.

van Koten, C., Gray, A.R., 2006. An application of Bayesian network for predicting object-oriented software maintainability. Inf. Softw. Technol. 48, 59–67. https://doi.org/10.1016/j.infsof.2005.03.002.

Venkatraman, K., Akashvarma, M., Siddharth, S., 2024. Enhancing software test effort estimation using ensemble learning algorithms. In: 2024 4th International Conference on Intelligent Technologies (CONIT). Presented at the 2024 4th International Conference on Intelligent Technologies (CONIT), pp. 1–5. https://doi.org/10.1109/CONIT61985.2024.10626442.

Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Van Mulbregt, P., 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. Nat. Methods 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2.

Wilcoxon, F., 1945. Individual comparisons by ranking methods. Biom. Bull. 1 (6), 80–83. https://doi.org/10.2307/3001968.

Yürüm, O.R., Ünlü, H., Demirörs, O., 2024. Towards the construction of a software benchmarking dataset via Systematic Literature Review. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). Presented at the 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 194–200. https://doi.org/10.1109/SEAA64295.2024.00037.

Zhang, D., Tsai, J.J.P., 2003. Machine learning and software engineering. Softw. Qual. J. 11, 87–119. https://doi.org/10.1023/A:1023760326768.

Ozan Raşit Yürüm is an assistant professor in the Department of Computer Engineering at İzmir Bakırçay University, Türkiye. He received his Master's and Ph.D. in Information Systems from Middle East Technical University. His research interests include agile software development, machine learning, programming languages, software quality management, software measurement, social software engineering, learning analytics, and educational data mining. Contact him at ozan.yurum@bakircay.edu.tr.

Hüseyin Ünlü is a researcher in the Computer Engineering Department at İzmir Institute of Technology, Türkiye. His research interests include software measurement, microservices, and software process improvement. He received his BSc. in computer engineering and MSc. in sustainable environment and energy systems from Middle East Technical University, and Ph.D. in computer engineering from Izmir Institute of Technology. Contact him at huseyinunlu@iyte.edu.tr.

Onur Demirörs is a professor in the Computer Engineering Department at İzmir Institute of Technology, Türkiye. His research areas include software process improvement, software measurement, software project management, conceptual modeling, and microservices. He received his MSc. Degree from Southern Methodist University, Dallas, Texas, USA, in 1993. He received his Ph.D. degree from SMU in 1995. Contact him at demirorso@gmail.com.